

דרישות פרויקט גמר בקורס פיתוח אפליקציות אינטרנטיות - סמסטר קיץ

כללי

על הסטודנטים לממש מערכת אינטרנטית לניהול, עריכה ופרסום חדשות, בדומה לאתרי חדשות מודרניים, בהתאם לדרישות המפורטות להלן ותוך שימוש אך ורק בטכנולוגיות שנלמדו במסגרת הקורס.

דרישות מנהליות

1. מבנה הקבוצות

- העבודה תתבצע בקבוצות של 4-5 סטודנטים.
- כל סטודנט בקבוצה חייב להכיר את כלל קוד המערכת ולהיות מסוגל לענות על שאלות הנוגעות לכל רכיב ורכיב במימוש.

2. ניהול גרסאות (Git)

- חובה להשתמש במערכת ניהול גרסאות **Git** לאורך כל תקופת הפיתוח.
- כל סטודנט נדרש לעבוד מהמחשב האישי שלו ובאמצעות חשבון **Git** פרטי.
- השימוש ב-**Git** יבוצע לפי אסטרטגיות מקובלות: **commits** תכופים ומתועדים, שימוש ב-**branches**, ביצוע **pull requests** ו-**merges**, וניהול תהליך עבודה שיתופי תקין.
- על הקבוצה להציג בהגנה תיעוד ברור של אופן העבודה המשותפת, לרבות תרומתו של כל סטודנט.

3. חלוקת סטודנטים לפי מרצים

- כל חברי הקבוצה חייבים להיות רשומים לאותו מרצה.
- לא ניתן לשלב סטודנטים הרשומים למרצים שונים.
- קבוצות שבהן התגלה ערבוב מרצים - הפרויקט שלהן לא ייבדק ויקבל ציון **0**.

4. מועדי ההגשות וההגנות

- את הפרויקט יש להגיש עד **24** שעות לפני מועד ההגנה. תיבת ההגשה תיסגר בשעה **23:59** ביום שלפני ההגנה.
- סטודנט שלא הגיש את הפרויקט במועד - יופחתו **20** נקודות מציון הפרויקט.
- מועד מיוחד יינתן באישור מיוחד של הממונה לענייני סטודנטים בלבד, בצירוף מסמכים ותיעוד מתאימים.
- כל סטודנט רשאי להיבחן פעם אחת בלבד; לא ניתן לשפר או לתקן לאחר שנבחנתם.

5. אופן ההגשה

- יש להגיש קובץ **Zip** של הקוד המלא במודל.
- בנוסף, יש לצרף בהערת ההגשה קישור מאובטח למאגר ה-**Git** של הפרויקט. המאגר חייב להיות פתוח לצפייה.
- חל איסור מוחלט להעלות למאגר קבצים המכילים מידע רגיש כגון **API keys** או טוקנים.

דרישות ההגנה

1. הצגת המערכת

- בהגנה תיבדק המערכת על בסיס גרסת הקוד שהוגשה בלבד, ללא אפשרות לבצע שינויים.
- המרצה יתפעל את המערכת בפועל ויבחן את יציבותה, שלמותה ותפקודה.

2. ידע בקוד

- המרצה רשאי לשאול כל סטודנט שאלות הנוגעות למבנה המערכת, לקוד שנכתב ולתפקידו.
- על כל סטודנט להפגין שליטה מלאה בקוד כולו, ולא רק בחלק בו עסק ישירות.

3. בדיקות נוספות

- מעבר למפגש ההגנה, המרצה יבצע בדיקה עצמאית של הפרויקט כולו לצורך קביעת הציון הסופי.

4. קביעת הציון

- הציון ייקבע על פי: ההגנה, איכות הקוד, נראות המערכת, חוויית המשתמש, יציבות, עבודה צוותית, ועמידה בדרישות הטכניות.

דרישות טכנולוגיות

1. טכנולוגיות מותרות

- יש להשתמש אך ורק בטכנולוגיות, בשפות ובארכיטקטורות שנלמדו בקורס (כגון: **CSS**, **HTML**, **JavaScript**, **Flexbox**, **Node.js**, **Express**, **MongoDB** בהתאם ללימוד).
- חל איסור להשתמש בספריות או פריימוורקים חיצוניים שלא נלמדו בכיתה, לדוגמה: **Angular**, **React**, **Vue** וכדומה.

2. למידה עצמית

- חלק מדרישות הפרויקט מהוות דרישות ללמידה עצמית, עליהן תיבחנו.
- נדרשת היכולת להיעזר בידע שנרכש בכיתה וליישמו באופן עצמאי לפתרון בעיות חדשות.

דרישות נוספות

1. מקוריות הקוד

- הקוד חייב להיות פרי עבודת הסטודנטים בקבוצה בלבד.
- יתבצעו בדיקות אוטומטיות וידניות לאיתור קוד מועתק. קוד מועתק יוביל לפסילת הפרויקט כולו.

2. שימוש בבינה מלאכותית

- מותר להיעזר בכלים מבוססי בינה מלאכותית (כגון **ChatGPT** או דומים) במהלך הפיתוח.
- עם זאת, חובה על כל סטודנט להבין לעומק את הקוד שנוסף באמצעות כלים אלו.
- במהלך ההגנה, המרצה רשאי לשאול שאלות על כל שורה בקוד. חוסר יכולת להסביר את הקוד ייחשב כחוסר שליטה בפרויקט ועלול להשפיע מהותית על הציון.

3. תיעוד

- חובה לצרף מסמך תיעוד (**README**) הכולל:

1. הוראות התקנה והרצה של המערכת.
2. מבנה הפרויקט (תיקיות וקבצים עיקריים).
3. תיאור קצר של הפונקציונליות המרכזית והפיצרים המיושמים.

4. חוקיות ושימושים מותרים

- ניתן להשתמש אך ורק בכלים ותכנים חוקיים, חינוכיים ומותרים לשימוש.

5. תצוגה ובדיקה מקדימה

- על הסטודנטים לוודא שהמערכת פועלת על מחשביהם האישיים וכי ניתן להריץ ולהדגים אותה בהצלחה טרם ההגנה.

פרויקט מסכם - מערכת חדשות "The Daily Web"

אפיון מוצרי

מסך הבית - פיד החדשות

העמוד הראשי פתוח לכל המשתמשים ויציג כתבות שאושרו ופורסמו בלבד.

העמוד יכלול:

- פיד חדשות עם גלילה אינסופית, שבו ייטענו אוטומטית 20 כתבות נוספות כאשר המשתמש מתקרב לתחתית הפיד
- חיפוש כתבות
- אפשרות לסינון לפי קטגוריה ו/או נצפה-לא נצפה, ולמיון לפי תאריך פרסום או פופולריות
- הצגת מידע בסיסי לכל כתבה: כותרת, תמונה, תקציר, קטגוריה, כתב ותאריך פרסום
- מעבר לעמוד המלא של הכתבה
- חיפוש, סינון, מיון וטעינת כתבות נוספות יתבצעו ללא רענון מלא של העמוד

עמוד כתבה

עמוד כתבה יהיה פתוח לכל המשתמשים ויכלול:

- פרטי הכתבה: כותרת, שם הכתב, קטגוריה, תאריך פרסום ותמונה ראשית
 - תוכן הכתבה המלא
 - אזור תגובות הכולל את רשימת התגובות וטופס להוספת תגובה חדשה
 - עם הוספת תגובה חדשה, היא תופיע מיד ברשימת התגובות, ללא טעינה מחדש של הרשימה כולה
- כל כניסה לכתבה תילקח בחשבון לצורך נתוני הצפייה והסטטיסטיקות של המערכת.

הגבלת תגובות

כדי למנוע ספאם:

- אורח יכול לפרסם לכל היותר 3 תגובות בדקה מאותו מכשיר
- ניסיון לעבור את המגבלה ייחסם על ידי השרת ויחזיר הודעה מתאימה למשתמש

מסך התחברות

המערכת תכלול מסך התחברות עבור כתבים ועורכים.

לאחר התחברות:

- כתב יועבר לאזור העבודה שלו
- עורך יוכל לגשת לאזור הניהול
- הרשאות המשתמש ייקבעו לפי המשתמש המזוהה ולא לפי מידע שניתן לשנות ידנית בדפדפן

אזור כתב - ניהול כתבות

כתב יראה את הכתבות השייכות לו ואת מצבן.

כתבה יכולה להיות באחד מהמצבים הבאים: **"בהכנה"**, **"ממתינה לאישור עורך"**, **"פורסמה"**, **"הוחזרה לתיקונים"**.

הכתב יוכל:

1. ליצור כתבה חדשה
2. לערוך כתבה שלו שנמצאת בהכנה
3. להעביר כתבה שהושלמה לאישור עורך
4. לראות הערה שכתב עורך כאשר כתבה הוחזרה לתיקונים
5. לבצע את התיקונים ולהגיש אותה מחדש
6. לערוך גם כתבה שכבר פורסמה

מעברים בין מצבי כתבה

המעברים בין מצבי הכתבה יהיו בהתאם לתהליך הבא:

- כתבה חדשה נוצרת במצב **"בהכנה"**
- כתב רשאי להעביר כתבה השייכת לו מ-**"בהכנה"** ל-**"ממתינה לאישור עורך"**
- עורך רשאי להעביר כתבה מ-**"ממתינה לאישור עורך"** ל-**"פורסמה"**, או להחזיר אותה למצב **"הוחזרה לתיקונים"** בצירוף הערה
- כתב רשאי לערוך כתבה שהוחזרה לתיקונים ולהעביר אותה מחדש ל-**"ממתינה לאישור עורך"**
- כל מעבר אחר בין המצבים אינו מותר

כאשר נערך תוכן של כתבה שכבר פורסמה, הגרסה החדשה תעבור את אותו תהליך אישור, בעוד שהגרסה האחרונה שאושרה תמשיך להיות מוצגת לציבור בהתאם לדרישות המפורטות בהמשך.
אין דרישה לתמיכה בעריכה בו-זמנית של אותה כתבה על ידי מספר משתמשים, וניתן להניח שמצב כזה אינו מתרחש.

המשכיות עבודה

בזמן כתיבת או עריכת כתבה:

- עבודתו של הכתב תישמר באופן שוטף ללא צורך בלחיצה יזומה על כפתור "שמור"
- סגירת הדפדפן, רענון העמוד או מעבר למחשב אחר לא יגרמו לאובדן העבודה
- כאשר הכתב חוזר לעריכה, עליו לקבל את הגרסה האחרונה שעליה עבד

עריכת כתבה שכבר פורסמה

כאשר כתב מתחיל לערוך כתבה קיימת שכבר פורסמה:

- ציבור הקוראים ימשיך לראות את הגרסה האחרונה שאושרה
- השינויים של הכתב לא יופיעו באתר הציבורי בזמן העבודה
- גם לאחר שליחת השינויים לאישור, הכתבה הקיימת תמשיך להיות מוצגת כרגיל
- רק לאחר אישור עורך יופיע התוכן החדש לציבור

אזור עורך - ניהול ואישור תוכן

העורך יוכל לצפות בכלל הכתבות במערכת ולסנן אותן לפי מצבן.

עבור כתבה שממתינה לאישור, העורך יוכל:

1. לצפות בתוכן שהוגש
2. לערוך את הכתבה בעצמו
3. לאשר ולפרסם את הכתבה
4. להחזיר את הכתבה לכתב לצורך תיקונים
5. לצרף הערה המסבירה אילו תיקונים נדרשים
6. למחוק כתבה בהתאם לצורך

כאשר מדובר בעדכון לכתבה שכבר פורסמה, העורך צריך להיות מסוגל להבין מהו התוכן המפורסם כרגע ומהו התוכן החדש שממתיין לאישור.

אישור השינויים יהפוך את התוכן החדש לגרסה המוצגת לקוראים.

אזור סטטיסטיקות - Impact Analytics

העורך יוכל לבחור כתבה ולצפות בגרף המתאר את כמות הצפיות בכתבה לאורך זמן.

ניתן להשתמש ב-**Chart.js** או ב-**canvas** לצורך הצגת הגרף.

הגרף יכלול:

1. ציר זמן
2. כמות צפיות לאורך ציר הזמן
3. סימון ברור של נקודות הזמן שבהן עורך אישר ופרסם עדכון לכתבה
4. אפשרות להבין מהגרף כיצד השתנתה כמות הצפיות לפני ואחרי פרסום עדכון

לדוגמה, אם כתבה עודכנה בשעה **14:00**, ניתן יהיה לראות על אותו גרף את התנהגות הצפיות לפני ואחרי נקודת העדכון.

יש להניח שהאתר עשוי לשרת אלפי קוראים במקביל. עליכם להחליט כיצד נכון לאסוף, לייצג ולשמור את הנתונים האלו כדי לתמוך בגרף המבוקש.

ארכיטקטורה וטכנולוגיות

- השרת יתבסס על **Node.js** עם **Express** כמסגרת עבודה.
- המערכת תתוכנן ותמומש לפי דפוס **MVC**, עם הפרדה ברורה בין **View**, **Model** ו-**Controller**.
- אחסון ושליפת הנתונים יתבצעו באמצעות **MongoDB** ו-**Mongoose**.
- בצד הלקוח ייעשה שימוש ב-**Vanilla JavaScript**, כולל **Ajax** לביצוע פעולות אסינכרוניות מול השרת ללא רענון מלא של העמוד
- ממשקי המידע של המערכת יתוכננו לפי עקרונות **REST**
- המערכת תעשה שימוש ב-**EJS** במקומות המתאימים
- עמוד כתבה חייב להיות נגיש למנועי חיפוש: התוכן המלא של הכתבה יופיע כבר ב-**HTML** הראשוני המוחזר מהשרת ולא יהיה תלוי בהרצת **JavaScript** בדפדפן
- במקביל, אזורי אינטראקטיביים כגון חיפוש, סינון, דפדוף ותגובות יתעדכנו ללא רענון מלא של העמוד
- המערכת תעשה שימוש ב-**HTML5** ובתגיות סמנטיות מתאימות
- הממשק יהיה רספונסיבי ויתאים לעבודה במחשב, טאבלט ומכשיר נייד

מסד נתונים ומודלים

- המערכת תכלול לפחות ארבעה מודלים עיקריים:

a. משתמשים

b. כתבות

c. תגובות

d. נתוני צפייה וסטטיסטיקות

- כל מודל יתמוך בפעולות **CRUD** (מלאות: **Create, Read (List/Search), Update, Delete**)
- מנגנון החיפוש יאפשר חיפוש לפחות על שדה מרכזי אחד (למשל לפי כותרת הכתבה)

אבטחה, הזדהות והרשאות

המערכת תכלול שלושה סוגי משתמשים: אורח (**Guest**), כתב (**Reporter**), עורך (**Editor**)

הזדהות כתבים ועורכים תבוצע באמצעות שם משתמש וסיסמה.

- סיסמאות לא יישמרו במסד הנתונים כטקסט גלוי ולא יהיה ניתן לשחזר מהן את הסיסמה המקורית
- משתמש שאינו מחובר יוכל לגשת רק לאזורים הציבוריים של האתר
- כתב יוכל ליצור כתבות ולערוך רק את הכתבות השייכות לו
- כתב אינו יכול לפרסם כתבה בעצמו
- עורך יכול לצפות ולערוך את כלל הכתבות במערכת, לאשר פרסום, להחזיר כתבה לתיקונים ולמחוק תוכן
- כל בדיקות ההרשאות יתבצעו גם בצד השרת. הסתרת כפתורים או מסכים בצד הלקוח אינה נחשבת למימוש הרשאות

אם השרת מבצע **Restart**, משתמש שכבר הזדהה למערכת צריך להיות מסוגל להמשיך להשתמש בה כרגיל, ללא צורך בהתחברות מחדש.

טיפול בשגיאות, תקינות וביצועים

- יש לטפל בשגיאות, מקרי קצה ונתונים לא תקינים הן בצד הלקוח והן בצד השרת
- פעולות שאינן מורשות או נתונים שאינם תקינים לא יגרמו לקריסת השרת
- חיפוש, סינון ומיון חייבים להחזיר תוצאות נכונות
- המערכת צריכה להישאר מהירה ושמישה גם כאשר מאגר הנתונים מכיל אלפי כתבות
- יש לשמור לוגים של שגיאות ואירועים תפעוליים משמעותיים

אינטגרציה עם שירות חיצוני

בסיידבר של האתר יוצג ווידג'ט מזג אוויר.

- יש להשתמש ב-**Web Service** חיצוני חינמי מוכר, לדוגמה **OpenWeatherMap** או שירות מקביל
- יש להשתמש במסלול שאינו דורש הזנת פרטי אשראי
- נתוני מזג האוויר המוצגים למשתמש יכולים להיות בפיגור של עד 15 דקות לכל היותר
- יש להניח שהאתר עשוי לשרת אלפי משתמשים במקביל

הדגמה

לפני ההגנה יש להזין למערכת נתוני דמה המאפשרים להציג את כלל הדרישות בצורה מלאה.

נתוני ההדגמה יכללו לפחות:

- 500 כתבות במצבים ובקטגוריות שונות
 - מספר משתמשים מסוג כתב
 - משתמש עורך
 - תגובות לכתבות
 - כתבות שנמצאות בתהליך עבודה
 - כתבות שממתינות לאישור
 - כתבות שפורסמו
 - לפחות מספר כתבות שעברו כמה עדכונים לאחר הפרסום
 - נתוני צפייה לאורך זמן בכמות מספקת להצגת גרף ה-**Impact Analytics** ונקודות פרסום העדכונים
- במהלך ההגנה יש להדגים גם מקרי הרשאה, המשכיות עבודה, עבודה לאחר **Restart** של השרת, התנהגות החיפוש והדפדוף על מאגר הנתונים, הגבלת תגובות והתנהגות המערכת מול שירות מזג האוויר.